# 1. Detailed Implementation Logic

In xv6, every process has a cwd pointer in its struct proc that points to a struct inode. However, an inode only knows its own ID; it doesn't store its own name or its full path. To implement **pwd**, you had to create a **"bottom-up" traversal logic**.

- **The getcwd System Call:** You created this in the kernel (sysproc.c) to traverse the directory tree. It starts at the current inode (.), opens the parent directory (..), and searches the parent's entries to find which name matches the child's inode number.
- **Recursive Climbing:** This process repeats—moving from the current directory to its parent—collecting names until it reaches the **root inode** (where . and .. are the same).
- **Kernel-to-User Transfer:** Once the full path is determined, the kernel uses copyout() to safely move that string into the memory buffer provided by the user program.
- **Infrastructure Changes:** You assigned a unique system call number in syscall.h, mapped it in syscall.c, and defined the user-space interface in user.h and usys.S.

---

# 2. How to Execute

To demonstrate this for Project 1, you added a user program called pwd.c to the xv6 source code.

1. **Add the User Program:** You placed pwd.c in the user/ directory. This program calls your new getcwd() system call and prints the result to the console.
2. **Update the Makefile:** You added _pwd to the UPROGS list in the xv6 Makefile so it compiles into the filesystem image.
3. **Compile and Run:**
    - Run make qemu to start the xv6 environment.
    - Inside the xv6 shell, use cd to navigate into a directory (e.g., cd a).
    - Type pwd and hit enter.
4. **Verification:** The terminal should display the full path (e.g., /a/b), proving the system call correctly traversed the inodes back to the root.

---

# 3. Documentation Paragraph

"For Project 1, I implemented the getcwd system call and the pwd user utility to provide absolute path resolution within the xv6 environment. My implementation involved modifying the kernel to perform a reverse traversal of the file system hierarchy, starting from the current process's cwd inode and iteratively searching parent directories (..) for matching inode numbers to reconstruct the directory names. This logic required updates to syscall.c, sysproc.c, and the xv6 Makefile to integrate the new call and the user-level demonstration program. The final utility was verified by navigating through multiple directory levels in the xv6 shell and

successfully printing the reconstructed absolute path, as shown in the attached execution screenshots."

```
$ pwd
/
$ cd pwd_test
$ pwd
/pwd_test
$ ls
.                1 33 32
..               1 1 1024
$ mkdir test_dir
$ cd test_dir
$ pwd
/pwd_test/test_dir
$
```